

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

Este documento detalla la configuración inicial y la metodología de trabajo asistido con experiencia en plataformas de alto nivel. Con conocimientos en Django, Firebase, HTML, Java y el ecosistema de Visual Studio.

Contenido

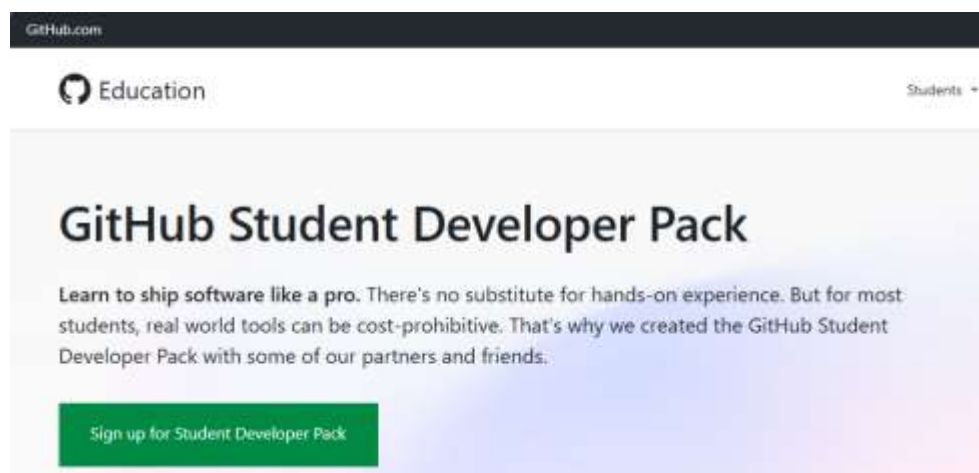
Fase 1: Configuración del Entorno de Desarrollo	1
Fase 2: Metodología de Apoyo con Inteligencia Artificial.....	3
Fase 3: Estructura y Explicación de Carpetas del Proyecto	5
Fase 4: Requisitos e Instalación del Entorno (Checklist).....	11
Fase 5: Ejecución Local, Pruebas y Control de Calidad	14
Fase 6: Flujo de Trabajo en Repositorios y Fusión de Código	16

Fase 1: Configuración del Entorno de Desarrollo

Para comenzar a trabajar, es indispensable preparar las herramientas y los accesos necesarios.

1. Obtener GitHub Copilot Pro

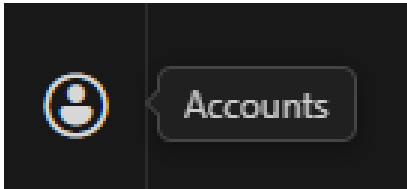
- Ingresa al portal educativo: [GitHub Student Developer Pack](#).
- Registra tu cuenta y sigue las instrucciones de verificación con tu correo institucional para activar los beneficios de estudiante, lo que incluye el acceso a Copilot Pro.



2. Vincular la cuenta en Visual Studio

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

- Abre Visual Studio en tu equipo.
- Inicia sesión utilizando la cuenta de GitHub con la que previamente activaste Copilot Pro. Esto garantizará que las herramientas de autocompletado y el chat de asistencia estén **habilitados** en tu entorno de trabajo.



Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

Fase 2: Metodología de Apoyo con Inteligencia Artificial

Aviso Importante: Las siguientes instrucciones están diseñadas para facilitar el desarrollo guiado. **En caso de tener conocimiento avanzado sobre la tarea a realizar, omite esta sección** y procede a programar directamente.

Si necesitas la asistencia de IA, sigue este flujo de trabajo y copia y pega los siguientes **prompts optimizados**:

1. Estructuración y Arquitectura (IA Externa: ChatGPT, Gemini, DeepSeek, etc.)



Antes de tirar código, utiliza una IA externa para traducir tus ideas en una arquitectura de software sólida.

- **Instrucción:** Inicia un nuevo chat y envía **exactamente** el siguiente texto para configurar a la IA como tu Arquitecto de Software:

Tu tarea es leer mis ideas (que te daré con mis propias palabras) y traducirlas de manera profesional a una estructura de proyecto. **No generes código fuente todavía.** Limitate exclusivamente a diseñar la arquitectura, definir los nombres correctos de los archivos, organizar las carpetas y establecer la lógica del sistema basándote estrictamente en lo que te pido."

3. Desarrollo Asistido en Visual Studio (**Claude Sonnet 2.5 - 4.5**)

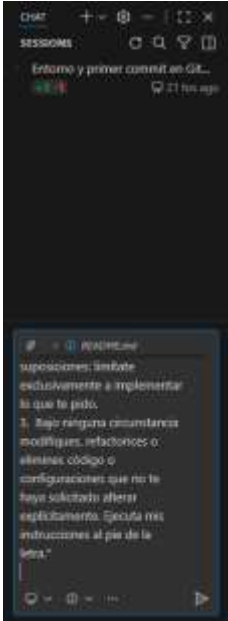
Claude Sonnet 4	1x
✓ Claude Sonnet 4.5	1x
Claude Sonnet 4.6	1x

Una vez que tengas la estructura definida y vayas a generar el código, utilizarás el asistente integrado en tu editor de código.

- **Instrucción:** Asegúrate de seleccionar el modelo **Claude Sonnet** en tu entorno.
- **Preparación:** Antes de empezar a trabajar en los archivos o pedirle código, envíale este prompt para establecer **límites estrictos**:

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

"Actúa como un Desarrollador Full-Stack Experto en la creación de páginas web. Antes de empezar, analiza y estudia todo el contexto y la estructura actual del proyecto.



Sigue estas reglas estrictamente:

1. Escribe código limpio, profesional y funcional.
2. Haz caso omiso a suposiciones: límitate exclusivamente a implementar lo que te pido.
3. **Bajo ninguna circunstancia** modifiques, refactorices o elimines código o configuraciones que no te haya solicitado alterar explícitamente. Ejecuta mis instrucciones al pie de la letra."

3. Resolución de Errores Complejos (ChatGPT 4.0 - 5.0)

Es normal encontrarse con fallos técnicos o "bugs" que el modelo del editor no logre solucionar de inmediato. Para esto, sacaremos el error del editor para analizarlo en frío.

- **Instrucción:** Si Claude Sonnet falla en darte la solución, dirígete a ChatGPT y utiliza esta plantilla para resolver el problema:

"Actúa como un Ingeniero de Soporte y Depuración de Software Senior. Estoy trabajando en un proyecto web y me he encontrado con un error crítico que mi asistente principal no pudo resolver.

Contexto de lo que intento hacer: [Describe brevemente qué intentabas lograr] **El error exacto es el siguiente:** [PEGA AQUÍ EL CÓDIGO DE ERROR O LOG DE LA CONSOLA]

Analiza el problema paso a paso, explícame brevemente la causa raíz del fallo y entrégame la solución exacta y el código corregido para implementarlo directamente."

GPT-5.1

1x

GPT-5.1-Codex

1x

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

Fase 3: Estructura y Explicación de Carpetas del Proyecto

A continuación, se detalla la arquitectura completa de **CodeKids Web**. Esta tabla contiene cada carpeta, sus archivos clave y su propósito específico para saber exactamente dónde modificar según la tarea asignada.

CodeKids-Clean

> .vscode

> app

> auth

> css

> Dashboard

> dev_artifacts_removed

> functions

> images

> js

> node_modules

> preview

> profesores

> scripts

> static

> tools

> Vistas_Publicas

> .gitignore

> CodeKidsMultiRoot.code-workspace

> firebase-config.js

> firebase.json

> firestore.indexes.json

> firestore.rules

> index.html

> package-lock.json

> package.json

> Vistas_Publicas

> .gitignore

> CodeKidsMultiRoot.code-workspace

> firebase-config.js

> firebase.json

> firestore.indexes.json

> firestore.rules

> index.html

> package-lock.json

> package.json

> PRESENTACION_CODEKIDS.md

> service-account-dev.json

> start-intendente.ps1

> start.ps1

> storage.rules

OUTLINE

TIMELINE

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

Categoría / Carpeta	Archivos Clave que Contiene	Descripción Detallada y Cuándo Usarla
Raíz del Proyecto CodeKids-Clean/	index.html firebase.json, firestore.rules, firestore.indexes.json, storage.rules, firebase-config.js service-account-dev.json package.json, package-lock.json, node_modules/ CodeKidsMultiRoot.code-workspace, .vscode/ start.ps1, start-intendente.ps1	Aquí están las configuraciones globales y el “esqueleto” del proyecto: • index.html: Punto de entrada principal del frontend (estructura base). • Archivos Firebase: Definen hosting, reglas de seguridad de Firestore y Storage, y la configuración del cliente. • Credenciales: service-account-dev.json es para el entorno de desarrollo (solo backend/scripts, tratar con cuidado). • Dependencias Node: Herramientas y scripts del proyecto raíz (no el frontend en sí). • Configuración VS Code: Para el workspace multi-root. • Scripts PowerShell: Sirven para levantar o abrir partes específicas del proyecto rápidamente (ej. dashboard de intendente).

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

Frontend: Vistas Vistas_Publicas/	Páginas HTML (landing, info, etc.)	Páginas públicas accesibles sin login. Aquí se editan contenidos y la estructura de las páginas “abiertas” del sitio.
Frontend: Vistas Dashboard/	Subcarpetas: Dashboard_Administrador/ Dashboard_Estudiante/ Dashboard_Intendente/ Dashboard_Profesor/	Contiene los dashboards internos separados por rol. Cada subcarpeta suele tener el HTML principal y a veces assets específicos de ese rol en particular.
Frontend: Vistas auth/	register.html, etc.	Páginas relacionadas con la autenticación pública. Se modifica cuando cambian los formularios o el flujo de acceso/registro.
Frontend: Vistas profesores/	bienvenida/, etc.	Contenido específico para profesores. Se usa para páginas informativas o flujos específicos exclusivamente de este rol.
Frontend: Vistas preview/	teacher-dashboard.html, etc.	Prototipos y vistas de prueba. Sirve para experimentar con nuevos diseños sin afectar el dashboard “oficial” en producción.
Frontend: Estilos css/	style.css estudiante-kids.css intendente.css	Hojas de estilo del frontend separadas por módulos: • Base: style.css (estilos globales).

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

	dashboard-enhancements.css laboratorio-enhancements.css lecciones-enhancements.css tareas-enhancements.css	• Por Rol: estudiante-kids.css, intendente.css. • Mejoras (Enhancements): Estilos extra para partes específicas (panel principal, laboratorio, lecciones, tareas). <i>Un dev nuevo suele tocar aquí cuando cambia el diseño visual, colores, espaciados, etc.</i>
Frontend: Lógica js/	Core: main.js, app.js, auth.js, firebase-init.js, firebase-config.js UI/Funcionalidad: dashboard-enhancements.js, dashboard-lecciones.js, tareas-enhancements.js, lecciones-enhancements.js, chat-enhancements.js, global-ui.js, student-shell.js, theme-toggle.js Extras: gamification.js, cody-ollama.js Desarrollo: api-mock.js, dev-auth-mock.js, admin-demo.js	Scripts del lado del cliente organizados por funcionalidad: • Core: Inicialización general, lógica de login/registro, gestión de sesión y cliente Firebase en el navegador. • Comportamiento Avanzado: Todo lo terminado en -enhancements.js controla UI dinámica, filtros, layout y modo oscuro. • Extras: Lógica de puntos/logros (gamification) o integraciones de IA (cody-ollama). • Mocks: Utilidades de desarrollo y demostración.

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

		<i>Se modifica al cambiar la lógica de negocio o interacciones del usuario.</i>
Assets Estáticos images/ y static/images/	Subcarpetas: Fondos/, icons/, minicoders/	Recursos gráficos. Logos, fondos, iconos e imágenes preparadas para servir de forma estática. Aquí se agregan nuevos recursos visuales y se actualizan los existentes.
Backend: Servidor functions/	index.js, users.json, package.json, node_modules/, carpeta tools/	Proyecto Node separado para Firebase Cloud Functions: • index.js: Punto de entrada del backend (endpoints, triggers, lógica de servidor). • users.json: Datos de usuarios de ejemplo. <i>Aquí se toca cuando hay que crear/actualizar endpoints, triggers de Firestore o lógica de negocio segura.</i>
Scripts y Herramientas scripts/	create-admin.js, create-estudiante.js, create-profesor.js, create-intendente.js set-admin-claim.js, reset-profesor-password.js, update-auth-password.js, sync-password-resets.js	Scripts Node para administrar Firebase (sin navegador): • Creación: Scripts para crear usuarios de distintos roles.

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

	test-login.js, test-password-reset-request.js setup-admin-emulator.js, check-data.js, users_export.json, profesor-data.json	• Autenticación: Asignar claims de admin, resetear y sincronizar contraseñas. • Testing/Setup: Emuladores, verificación y exportación de datos. <i>Normalmente usados por personas en roles de DevOps o mantenimiento, no para UI.</i>
Scripts y Herramientas tools/	check_login_resources.py create_zip_backup.py createTestChats.html	Utilidades misceláneas para desarrollo: Chequear recursos de login, generar backups comprimidos o herramientas de prueba de chats. Útiles para automatizar tareas y debugging.
Otras Carpetas dev_artifacts_removed/	teacher-dashboard.old.html, admin.html.copy, etc.	Archivos históricos y depreciados. Se consultan para ver cómo era algo antes, pero en general no se editan para nuevas features.
Otras Carpetas app/	Proyecto_CodeKids.code-workspace	Workspace secundario. Usado habitualmente para abrir solo una parte concreta del código en VS Code; no es la app “en producción” separada.
Documentación Directorio Raíz	PRESENTACION_CODEKIDS.md	Documento de presentación del proyecto. Resumen de alto nivel para lectura de humanos.

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

Fase 4: Requisitos e Instalación del Entorno (Checklist)

Para que el proyecto funcione correctamente en tu equipo local, debes instalar y configurar una serie de herramientas **obligatorias**. Sigue este orden estrictamente para evitar errores de compatibilidad.

A excepción del paso 1, podemos descargar todo mediante la **terminal** de VSC.

1. Sistema Operativo y Terminal

- **Windows 10 o Windows 11.**
- **PowerShell:** Viene por defecto en Windows. Lo utilizarás para ejecutar los scripts de arranque rápido del proyecto (ej. start.ps1, start-intendente.ps1).

2. Editor de Código

- **Visual Studio Code (VS Code):** Es el editor estándar del proyecto. El repositorio ya **incluye** archivos de configuración (.code-workspace) optimizados para este entorno.



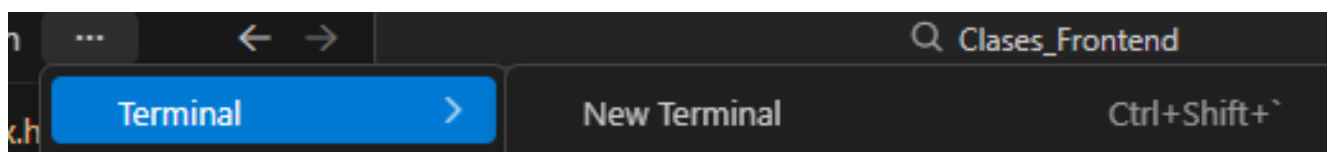
3. Motor de JavaScript (Node.js)

El backend de Firebase y los scripts de mantenimiento dependen de esto.

- **Node.js (Versión 20.x):** Es **obligatorio** instalar específicamente la versión 20 (puedes descargarla desde nodejs.org). El archivo package.json exige esta versión para compilar las Cloud Functions.
- **NPM (Node Package Manager):** Se instala automáticamente junto con Node.js.

Acción requerida tras instalar Node:

1. Abre tu **terminal** en la raíz del proyecto (CodeKids-Clean/) y ejecuta: npm install
2. Navega a la carpeta del backend (cd functions/) y ejecuta de nuevo: npm install



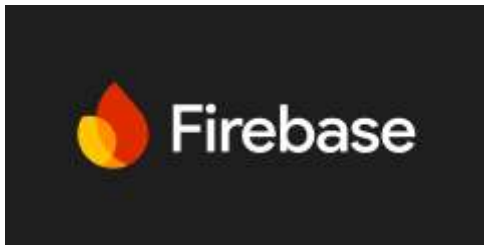
Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

4. Herramientas de Firebase (Firebase CLI)

Necesario para correr servidores locales (emuladores) y subir código a producción.

- Abre tu terminal y ejecuta el siguiente comando para instalarlo globalmente: `npm install -g firebase-tools`
- **Vinculación de Cuenta:** Ejecuta `firebase login` en tu terminal.

Nota Importante: El acceso al proyecto de Firebase y los archivos de credenciales confidenciales (como `service-account-dev.json`) **se te entregarán por un canal privado**. No intentes usar cuentas personales no autorizadas.



5. Motor de Inteligencia Artificial Local (Ollama)

El proyecto cuenta con un asistente de IA integrado ("Cody"). Para que funcione de manera local sin costos de API, necesitas tener el motor corriendo en tu equipo.

- **Instalar Ollama:** Descárgalo e instálalo desde su página oficial.
- **Descargar el Modelo:** Abre tu terminal y ejecuta: `ollama run llama3.2:3b`. Esto descargará el modelo específico que utiliza el código (`cody-ollama.js`).
- **Ejecución:** Siempre que vayas a probar la IA en el proyecto, el servidor de Ollama debe estar activo corriendo el comando: `ollama serve`.

6. Python (Solo para Herramientas Auxiliares)

Aunque no es obligatorio para correr la página web, es necesario si deseas usar los scripts de la carpeta `tools/`.

- **Python 3.x:** Descárgalo desde `python.org` o la Microsoft Store.
- Asegúrate de marcar la casilla "Add Python to PATH" durante la instalación para poder ejecutar utilidades como crear respaldos ZIP (`create_zip_backup.py`) o auditar recursos (`check_login_resources.py`)

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

Resumen Operativo: ¿Cómo levanto el proyecto para trabajar? Una vez que tienes todo lo anterior instalado:

1. Asegúrate de tener Ollama corriendo (ollama serve).
2. Desde la raíz del proyecto, inicia los servicios de Firebase con: `firebase emulators:start` (Esto levanta la base de datos, el backend y el hosting en tu computadora).
3. Abre tu navegador moderno (Chrome/Edge,etc) y entra a la dirección local que te arroje la terminal (usualmente `localhost:5000` o similar).

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

Fase 5: Ejecución Local, Pruebas y Control de Calidad

El proyecto **CodeKids Web** opera bajo una regla de oro: **Todo el desarrollo y las pruebas se realizan de forma estrictamente local.** **Bajo ninguna circunstancia** se debe ejecutar el proyecto directamente en línea durante la fase de desarrollo para evitar sobrescribir la base de datos o el código que ya tenemos funcionando en la nube (producción).

1. Cómo correr el proyecto (Entorno Local)

Para visualizar tus cambios de forma segura, tu computadora simulará ser el servidor usando los emuladores de Firebase.

Pasos de ejecución:

1. Abre una terminal y arranca el motor de IA local ejecutando: ollama serve.
2. Abre una segunda terminal en la raíz del proyecto (CodeKids-Clean/) y levanta los servidores locales con el comando: firebase emulators:start
3. La terminal te arrojará varias direcciones locales (generalmente http://127.0.0.1:5000 para la web y http://127.0.0.1:4000 para el panel de Firebase). Abre esas direcciones en tu navegador.

ADVERTENCIA CRÍTICA: Queda estrictamente prohibido utilizar el comando firebase deploy a menos que un administrador te haya dado la orden explícita de subir la versión final. Todo tu trabajo debe quedarse en localhost.

2. Protocolo Obligatorio de Pruebas

Si modificas una pantalla, **eres responsable de que siga funcionando.** No asumas que "el código no afectó nada más".

- **Validación de Pantalla:** Una vez que modifiques el diseño o la lógica de una sección (por ejemplo, el *Dashboard del Estudiante*), **debes probar todos los botones**, formularios y funciones de esa pantalla específica.
- **Objetivo:** Comprobar empíricamente que no se ha dañado ningún código importante, que la conexión con el emulador de la base de datos sigue activa y que la experiencia del usuario está intacta.

3. Documentación de Cambios (Reporte en PDF)

Cada tarea terminada o modificación importante debe estar respaldada por un documento. No se aceptarán cambios "invisibles".

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

- **Formato:** Debes entregar un archivo **PDF** documentando tu trabajo.
- **Contenido Visual:** El documento debe basarse en capturas de pantalla (fotos del "Antes" y "Después" de la interfaz).
- **Descripción:** Debes redactar con tus propias palabras qué fue lo que pasó, qué modificaste en la vista o qué nueva función agregaste.
- **Regla de Cero Código:** El PDF es un reporte visual y funcional. **No debes incluir bloques de código**, scripts ni rutas de programación en este documento; límitate a explicar el impacto en la plataforma.

—Viernes 12 de diciembre—

Mejoras visuales y funcionales

Figura 1: Se agrego a Cody en ambos formularios de inicio.

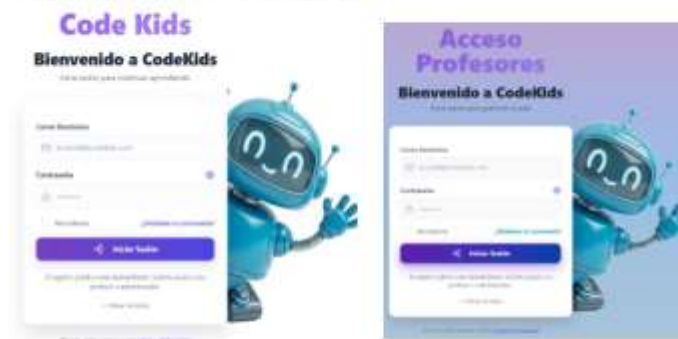


Figura 2: Se agregaron imágenes y animaciones a la tarjeta Creación de Foros.

Beneficios para Profesores
El sistema de foros te permite crear y gestionar tus propios foros de discusión y aprendizaje.



4. Generación de Recursos Visuales e Imágenes

Si durante tu desarrollo necesitas agregar nuevas imágenes de relleno, iconos, avatares o fondos para la plataforma, debes apoyarte en herramientas de Inteligencia Artificial generativa (como Midjourney, DALL-E, o la de tu preferencia).

Regla para Imágenes de IA: **Es obligatorio** revisar detalladamente cada imagen generada. **No debe aparecer ninguna marca de agua, texto extraño, ni el logo de la IA** en la imagen final. Si la imagen trae marca de agua, debes recortarla o generar una nueva antes de meterla a la carpeta images/ del proyecto.

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

Fase 6: Flujo de Trabajo en Repositorios y Fusión de Código

En **CodeKids Web** manejamos un flujo de trabajo específico para aislar las modificaciones antes de integrarlas al código principal. Es vital seguir estos pasos al pie de la letra.

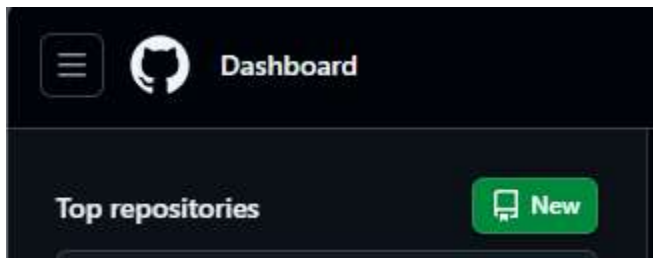
Actualmente, el desarrollo del proyecto está enfocado en **3 dashboards principales:**

- Estudiante
- Administrador
- Profesor

1. Obtención del Proyecto Base

Todo el desarrollo parte del código original alojado en GitHub.

- **Acceso:** El enlace al repositorio base se te entregará **por medio de un canal privado**.
- Debes descargar o clonar este repositorio en tu equipo para comenzar a trabajar y realizar las pruebas localmente.



2. Subida de Modificaciones (Nuevo Repositorio)

Una vez que hayas realizado tus cambios, verificado que las funciones operen correctamente y generado tu reporte en PDF, **NO** debes subir tu código al repositorio base.

- Debes crear un **nuevo repositorio** en tu propia cuenta de GitHub y subir tu versión modificada allí.
- **Nomenclatura obligatoria:** El repositorio debe nombrarse exactamente con la siguiente terminación: `Dashboard_[Rol_Modificado]_Correccion`

Ejemplos de nombres válidos:

- `Dashboard_Estudiante_Correccion`

Manual de Usuario: Primeros Pasos para el Proyecto "CodeKids Web".

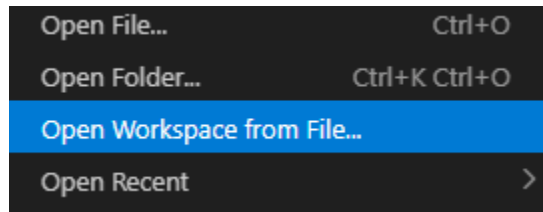
- Dashboard_Profesor_Correccion
- Dashboard_Administrador_Correccion

3. Cómo Sobrescribir el Proyecto Base

Cuando la modificación en el repositorio de corrección esté lista para integrarse al proyecto origen (CodeKidsWeb_Base), existen dos maneras oficiales de hacer la fusión:

Método 1: Sobrescritura Manual Consiste en abrir ambos proyectos simultáneamente y reescribir o trasladar manualmente el código nuevo hacia los archivos exactos del proyecto base.

Método 2: Workspace en Visual Studio y Asistencia de IA (Recomendado) Puedes utilizar el entorno de trabajo múltiple y tu asistente de IA (Claude Sonnet) para automatizar la extracción del código.



1. Abre un *Workspace* (Área de trabajo) en Visual Studio Code que contenga simultáneamente la carpeta del proyecto base y la carpeta del proyecto modificado.
2. Abre el chat de la IA en el editor y envíale exactamente este prompt, sustituyendo el nombre del dashboard y la sección específica que deseas migrar:

"Mi proyecto 'CodeKidsWeb_Base', es mi proyecto origen, y mi proyecto 'Dashboard_Estudiante_Correccion' (*cambiar por el que corresponda*), es el proyecto que quiero que extraigas solo la parte de -[INTRODUCIR SECCIÓN AQUÍ]- y la sobrescribas en el proyecto origen."